Experimental Evaluation of Scheduling Effects on TinyML Inference on ESP32 Using FreeRTOS

Anh Le
College of Sciences
Computer Science and Cybersecurity
Metropolitan State University
Saint Paul, MN, USA
lekhuenhatanh@gmail.com

Sena Houeto
College of Sciences
Computer Science and Cybersecurity
Metropolitan State University
Saint Paul, MN, USA
sena.houeto@metrostate.edu

***Abstract*—The integration of Tiny Machine Learning (TinyML) into resource-constrained embedded systems enables real-time, on-device intelligence for Internet of Things (IoT) applications. This paper presents the design and evaluation of a real-time inference system using TensorFlow Lite Micro on an ESP32-S3 platform running FreeRTOS. A task-based architecture is implemented to support periodic inference alongside concurrent workloads, with fine-grained instrumentation to measure latency, jitter, and worst-case execution time (WCET). The experimental evaluation is conducted under multiple scenarios, including baseline execution, periodic scheduling, and priority-induced interference. The results show that system load and task priority configurations significantly affect timing predictability, leading to increased latency variability and missed deadlines under contention. Quantitative analysis using deadline miss ratio, latency distribution, and WCET metrics demonstrates the challenges of maintaining real-time guarantees in embedded TinyML systems. The findings emphasize the importance of scheduling strategies and workload management in achieving reliable performance. This work provides a practical evaluation framework for assessing the real-time behavior of TinyML applications on embedded platforms. It highlights the importance of scheduling-aware system design for reliable edge intelligence.**

***Index Terms*—TinyML, real-time systems, FreeRTOS, embedded AI, scheduling, ESP32, latency, WCET**

I. Introduction

Recent advances in embedded machine learning have made it possible to execute inference directly on low-cost microcontrollers. This enables fast local decision-making in applications such as safety monitoring, industrial sensing, and intelligent IoT systems. Compared with cloud-based inference, on-device execution reduces communication delay, bandwidth dependence, and reliance on external connectivity. However, in real-time embedded environments, inference must compete with sensing, communication, and I/O tasks for limited processor time, which can significantly affect timing behavior.

This creates an important gap between model-level efficiency and system-level predictability. A model that appears lightweight in standalone evaluation may still exhibit latency inflation, jitter, or deadline misses once integrated into a multitasking embedded system. For real-time TinyML applications, understanding this gap is essential because the consequences of missed deadlines in embedded TinyML systems extend beyond degraded accuracy. In safety-critical applications, A delayed inference result may arrive too late to trigger a meaningful response, rendering the system unreliable regardless of model quality. For example, a wearable health monitor that fails to classify a sensor reading in time may miss a critical event. Likewise, an industrial anomaly detector that cannot keep up with its sampling rate becomes unreliable regardless of how accurate its model is. As this study shows, simply changing the priority of competing tasks can significantly increase the deadline miss rate. Thus, real-time correctness in embedded TinyML systems is as much a scheduling problem as a modeling challenge.

The objective of this work is to determine how scheduler-level interference affects the temporal predictability of TinyML inference on a resource-constrained platform. Specifically, this paper investigates the timing behavior of embedded TinyML inference under realistic multitasking conditions on an ESP32-S3 running FreeRTOS. The system executes periodic inference together with controlled background workloads so that the effects of scheduling contention and task-priority assignment can be measured directly. Fine-grained instrumentation captures latency, jitter, observed worst-case execution time (WCET), deadline misses, and lateness. The results show that timing behavior is strongly shaped by workload contention and priority configuration, and that these system-level factors can dominate performance even when the deployed model itself is compact.

The main contributions of this work are as follows:

- A FreeRTOS-based TinyML inference framework on the ESP32-S3 with fine-grained instrumentation for measuring latency, jitter, observed WCET, deadline misses, and lateness under controlled multitasking conditions.
- Experimental evidence that scheduler-level interference is the dominant factor limiting real-time TinyML performance, with deadline miss rates ranging from 1.5% under favorable scheduling to 59% under contention, and worst-case latency increasing from 23ms to over 350ms.
- Characterization of three distinct system behaviors as a function of task priority configuration, providing a practical schedulability boundary for embedded TinyML deployment.

II. Background

TinyML refers to the deployment of machine learning inference on resource-constrained embedded devices, typically microcontrollers with limited memory, computation, and power budgets. Lightweight runtimes such as TensorFlow Lite for Microcontrollers (TFLM) support this model through statically managed memory and low runtime overhead. In real-time embedded systems, correctness depends on both functional output and timing properties such as latency, jitter, worst-case execution time, and deadline compliance. FreeRTOS, a widely used real-time operating system for embedded platforms, employs a preemptive priority-based scheduler in which higher-priority ready tasks can interrupt lower-priority running tasks, while periodic execution is commonly implemented using `vTaskDelayUntil()`. Although these mechanisms support predictable execution, task interaction and background activity can still introduce substantial timing variation, making inference latency necessary to evaluate in the context of the full embedded workload rather than in isolation.

III. Related Work

Prior research relevant to this study spans three main areas: deployment of machine learning on embedded devices, timing behavior in real-time operating systems, and performance of embedded and IoT workloads under contention.

A. TinyML Deployment on Embedded Platforms

The growing feasibility of on-device inference has been driven by advances in compact model design, quantization, and lightweight runtimes. [1] shows that machine learning models can be deployed on microcontrollers with very limited memory and power budgets, while broader surveys of embedded artificial intelligence emphasize model compression, hardware-aware optimization, and accelerator support as key enablers [2]. However, this literature mainly evaluates deployment in terms of model size, inference accuracy, and energy efficiency, with less attention to timing behavior inside a multitasking real-time software stack.

B. Real-Time Scheduling and FreeRTOS Behavior

A substantial body of work has examined timing behavior in real-time systems, especially under fixed-priority scheduling. Classical studies on priority inversion and synchronization protocols show that task interaction can significantly affect blocking, response time, and overall schedulability [3], [4]. More recent work on FreeRTOS has analyzed scheduler behavior both formally and experimentally, showing that task interleaving, interrupt handling, and kernel overhead can produce nontrivial execution patterns in embedded systems [5], [6]. These studies provide important insight into the predictability limits of multitasking software, but generally focus on conventional real-time workloads rather than machine learning inference.

C. Embedded and IoT Workload Performance

Research on embedded and IoT system performance has also examined sensing, communication, and edge-processing workloads under realistic operating conditions, particularly in safety-critical monitoring and emergency-response settings [7], [8]. Related work has further studied the performance implications of lightweight IoT communication mechanisms such as MQTT in distributed edge-to-cloud environments [9], [10]. These studies are relevant because they reflect the mixed-workload conditions in which TinyML applications are often deployed. However, most focus on communication efficiency, message delivery, or general system responsiveness rather than the timing behavior of on-device inference as one task among several competing real-time activities.

D. Summary of Research Gap

Previous studies established that TinyML deployment on microcontrollers is feasible and that multitasking embedded systems can exhibit substantial timing variability under contention. However, there is still limited experimental evidence showing how task-priority assignment and competing workload activity affect inference latency, jitter, and deadline compliance in microcontroller-based TinyML systems.

IV. System Design

The system is designed around three principles: modular task decomposition, single-core execution to isolate scheduling interactions, and fine-grained instrumentation to support reproducible timing analysis.

A. System Architecture

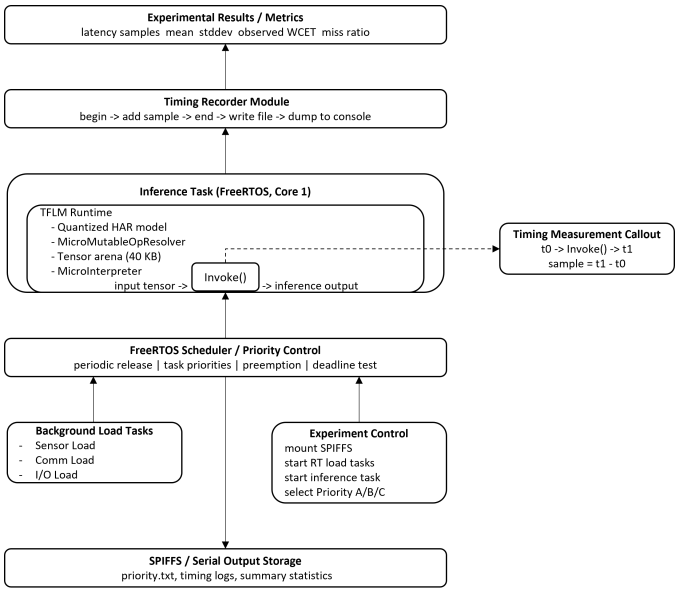


Fig. 1. Architecture of the proposed TinyML inference system on the ESP32-S3. The inference task runs under FreeRTOS scheduling alongside concurrent sensor, communication, and I/O tasks, with timing and logging support for runtime performance evaluation.

The system is implemented on an ESP32-S3 microcontroller, which provides a dual-core architecture suitable for real-time embedded applications. TFLM enables on-device machine learning inference, while FreeRTOS serves as the underlying real-time operating system, responsible for task scheduling and resource management. This modular decomposition reflects common embedded design practice and enables controlled experimentation under multitasking conditions. All inference processing is performed locally on the device. Figure 1 illustrates the overall system architecture, showing the interaction between the inference task, FreeRTOS scheduler, background workload tasks, and timing instrumentation.

B. Inference Execution Model

The inference functionality is implemented as a periodic real-time task. This task follows a time-triggered execution model using the `vTaskDelayUntil()` mechanism provided by FreeRTOS, which ensures consistent activation intervals and minimizes timing drift. Each execution cycle of the inference task consists of three sequential stages:

1) **Input Preparation**: Sensor or preloaded data is formatted into the input tensor required by the model. This step may include normalization, quantization alignment, or feature extraction.
2) **Model Inference**: The TFLM interpreter executes the neural network model using the prepared input tensor. This stage represents the core computational workload of the system.
3) **Output Handling**: The inference results are processed and optionally logged for analysis. This may include classification decisions, threshold evaluation, or storage of raw outputs.

The periodic execution model is critical for real-time systems, as it allows predictable scheduling behavior and facilitates the measurement of timing properties such as execution latency and WCET. By maintaining a fixed activation period, the system can be evaluated against deadline constraints and analyzed for temporal correctness.

The deployed model is based on the UCI Human Activity Recognition (HAR) dataset [11] and is converted to an INT8 quantized format for embedded execution. The model input consists of a 128×6 time-series window representing inertial sensor signals, and the output is a six-class prediction corresponding to walking, walking upstairs, walking downstairs, sitting, standing, and laying. The network uses a lightweight one-dimensional convolutional architecture suitable for microcontroller deployment, consisting of two Conv1D layers with 16 and 32 filters, respectively, each using a kernel size of 5 and ReLU activation, with MaxPooling1D layers after each convolution. The convolutional feature maps are then flattened and passed to a fully connected dense layer with 64 neurons and a final softmax output layer with 6 neurons. In total, the network contains 69,078 trainable parameters. To reduce memory footprint and improve inference efficiency on the embedded target, all weights and activations are quantized to 8-bit integers. Inference is performed using TFLM with a statically allocated tensor arena [1], [12].

C. Task-Based Workload Design

To emulate realistic embedded system conditions, the design incorporates multiple concurrent workload tasks that execute alongside the inference task. These tasks are intentionally designed to introduce controlled interference and resource contention. Each workload task represents a typical embedded function, such as sensor polling, communication handling, or I/O processing. Although simplified, these tasks capture the essential characteristics of real-world workloads:

- **Independence**: Each task operates independently of the inference task, allowing the scheduler to interleave execution based on priority and readiness.
- **CPU Utilization**: Tasks include computational loops that simulate processing load, thereby increasing CPU utilization and affecting scheduling behavior.
- **Cooperative Behavior**: Periodic yielding (e.g., via `vTaskDelay()`) is introduced to prevent starvation and maintain system stability.

This workload design enables the study of system behavior under multitasking conditions, where multiple tasks compete for CPU time. Such conditions are known to introduce challenges such as scheduling delays, jitter, and potential starvation in real-time systems [3].

D. Scheduling Model

The system employs the FreeRTOS preemptive priority-based scheduling model. Each task is assigned a fixed priority level, which determines its execution precedence. The scheduler operates according to the following principles:

- **Preemption**: Higher-priority tasks can interrupt and preempt lower-priority tasks at any time.
- **Priority Ordering**: Tasks are selected for execution based on their priority and readiness state.
- **Time Sharing**: Tasks with equal priority share CPU time according to the scheduling policy.

This scheduling model enables flexible experimentation with different priority configurations. By adjusting task priorities, the system can simulate various real-time scenarios, including favorable scheduling conditions and high-contention environments.

Such configurability is essential for evaluating the robustness of the inference task under different system loads and for analyzing phenomena such as deadline misses, latency variation, and priority inversion.

E. Timing Measurement Mechanism

To support detailed performance evaluation, the system incorporates a high-resolution timing measurement mechanism based on the `esp_timer_get_time()` API, which provides microsecond-level timestamps. Timing measurements are captured at key execution points within the inference task, including the start and end of each inference cycle, and

optionally intermediate stages. From these timestamps, several critical real-time metrics are derived:

- **Inference Latency**: The execution time of a single inference cycle.
- **Jitter**: The variability in execution time across multiple cycles.
- **Observed WCET**: The maximum observed execution time during the experiment.

All timing data is recorded and stored in a structured format on the device's file system (SPIFFS). To minimize measurement overhead and avoid perturbing system behavior, logging operations are lightweight and decoupled from critical execution paths.

This instrumentation framework enables accurate and reproducible analysis of system timing behavior, which is essential for evaluating real-time performance in embedded AI applications.

V. Experimental Setup

This section describes the hardware and software configuration, test scenarios, workload conditions, execution procedure, and performance metrics used to evaluate inference timing behavior under controlled multitasking conditions.

A. Hardware and Software Configuration

All experiments are conducted on an ESP32-S3 microcontroller that is running FreeRTOS. The system is developed using the ESP-IDF framework. Although the ESP32-S3 supports dual-core execution, all tasks are explicitly pinned to a single core (Core 1). This design choice ensures consistent contention among tasks and eliminates variability caused by parallel execution across cores. Also, to ensure comparability across runs, all experiments use the same processor and system configuration. As a result, the observed behavior reflects true scheduling interactions rather than hardware-level parallelism. Table I describes the task set and parameters.

TABLE I
Task parameters used in the experiments

Task	Period	CPU demand	Deadline
Inference	10 ms	one inference	10 ms
Sensor	10 ms	50,000-iteration CPU loop	10 ms
Communication	20 ms	80,000-iteration CPU loop	20 ms
I/O	50 ms	30,000-iteration CPU loop	50 ms

The inference task performs one TFLM inference per activation. The sensor task generates periodic short-duration CPU activity to emulate sampling and preprocessing load. The communication task introduces periodic communication-related processing load, and the I/O task performs periodic storage activity to emulate file-system interference.

B. Test Scenarios

To evaluate the impact of scheduling behavior on inference performance, three test scenarios are evaluated, as summarized in Table II. In all cases, the same set of background tasks is executed. Only the relative priority assignment of the inference task and competing workload tasks is changed across scenarios:

TABLE II
Task priorities for evaluated scheduling scenarios

Scenario	Inference	Sensor	Communication	I/O
Priority-A	5	3	3	2
Priority-B	3	4	4	2
Priority-C	2	5	5	5

- **Priority-A (High-Priority Inference):** The inference task is assigned the highest priority in the system. This configuration minimizes scheduling interference and represents a near-ideal execution condition for real-time inference.
- **Priority-B (Contended Execution):** The inference task is assigned priority 3, while the sensor and communication tasks are both raised to priority 4 and the I/O task remains at priority 2. This configuration introduces substantial contention between inference and background processing, representing a more stressed multitasking environment.
- **Priority-C (Starvation Scenario):** The inference task is assigned the lowest priority among the evaluated configurations (priority 2), while the sensor, communication, and I/O tasks are all assigned priority 5. This configuration induces severe scheduling interference and is intended to evaluate system behavior under extreme contention, including the possibility of starvation.

Each scenario is executed for 1000 consecutive inference iterations to ensure statistical reliability.

C. Workload Conditions

Workload tasks emulate common embedded functions such as sensor processing, communication handling, and I/O operations. Each task executes controlled computational loops to consume CPU cycles and periodically yields via `vTaskDelay()` to prevent starvation while maintaining measurable interference. The three scenarios allow to capture realistic IoT deployment conditions where inference must coexist with competing embedded activities.

D. Performance Metrics

The system is evaluated using a set of timing-based performance metrics that characterize its real-time behavior:

- **Inference Latency**: The execution time of each inference cycle, measured in microseconds. This represents the primary performance metric.
- **Jitter**: The variability in inference latency across multiple cycles, typically expressed as standard deviation or range. High jitter indicates unstable execution behavior.
- **Observed WCET:** The maximum observed execution time during the experiment. This is critical for assessing worst-case timing guarantees.
- **Deadline Compliance Rate**: The percentage of inference cycles that complete within a predefined deadline. This metric directly reflects the system's ability to meet real-time constraints.

- **Lateness**: The amount of time by which a task exceeds its deadline (if applicable), providing insight into the severity of deadline violations.

Together, these metrics provide a comprehensive view of system performance, capturing both average behavior and worst-case conditions.

E. Data Collection and Analysis

During execution, all timing data is recorded in real time using a high-resolution timer and stored in the device's SPIFFS file system. Each inference cycle generates a timestamp entry, forming a dataset of execution times.

The collected data is exported and analyzed offline using statistical tools. The analysis includes:

- Computation of mean, minimum, and maximum execution times
- Standard deviation and jitter analysis
- Percentile-based WCET estimation (e.g., 95th and 99th percentiles)
- Deadline miss counting and lateness distribution

This structured data collection approach enables reproducible experimentation and supports quantitative comparison across different test scenarios.

By combining controlled experimental conditions with detailed timing analysis, the setup provides a robust framework for evaluating the real-time performance of TinyML inference on embedded systems.

VI. Results and Analysis

A. Priority-A Scheduling Configuration

In the Priority-A experiment, the inference task was assigned a higher priority than the background sensor, communication, and I/O tasks to evaluate whether favorable scheduling could preserve real-time inference performance under concurrent load. The experiment was conducted over 1000 inference iterations.

TABLE III
Timing Results for Priority-A Configuration

Metric	Value
Mean latency	8248.95 µs
Standard deviation	1175.20 μs
Minimum latency	8074 µs
Maximum latency (observed WCET)	23289 µs
Deadline	10000 µs
Deadline misses	15 / 1000 (1.50%)
Worst-case lateness	13290 µs

The timing results for the Priority-A configuration are summarized in Table III. The inference task achieved a mean latency of 8248.95 μs, remaining below the specified deadline of 10000 μs. The relatively moderate standard deviation of 1175.20 μs indicates that most executions are clustered around the nominal execution time of approximately 8 ms.

However, the maximum observed latency of 23289 μs significantly exceeds the deadline, resulting in 15 deadline misses out of 1000 executions (1.50%). These results demonstrate that while the system performs reliably under typical conditions, occasional long-tail latency spikes still occur, preventing strict real-time guarantees.

B. Priority-B Scheduling Configuration

In the Priority-B experiment, the priorities of competing background tasks were increased to intensify scheduling contention and evaluate the sensitivity of inference timing under heavier interference.

TABLE IV
Timing Results for Priority-B Configuration

Metric	Value
Mean latency	38363.46 µs
Standard deviation	41886.39 µs
Minimum latency	7527 μs
Maximum latency (observed WCET)	350920 µs
Deadline	10000 µs
Deadline misses	590 / 1000 (59.00%)
Worst-case lateness	340921 µs

The timing results for the Priority-B configuration are summarized in Table IV. Under increased scheduling contention, the mean latency rises sharply to 38363.46 μs, significantly exceeding the specified deadline. The large standard deviation of 41886.39 μs reflects extreme variability in execution time, with latency values ranging from near-nominal execution to over 350 ms.

The system recorded 590 deadline misses out of 1000 executions (59.00%), with a worst-case lateness of 340921 μs. These results indicate that the inference task is unable to meet real-time constraints under this configuration. Although some executions are still completed within the nominal range, the timing distribution becomes highly dispersed, and both average-case and worst-case performance degrade substantially under heavy interference.

C. Comparative Timing Analysis

A comparison between Priority-A and Priority-B underscores the strong influence of scheduling configuration on inference timing behavior. Under Priority-A, the inference task sustains a mean latency below the 10 ms deadline, with relatively limited variability apart from occasional outliers. This 10 ms deadline is selected to match the periodic activation interval of the inference task, which in turn is derived from the sensor sampling rate used in the system (i.e., the system is designed to process new sensor data every 10 ms). This value represents a practical real-time requirement for time-sensitive embedded applications. Although the deadline can be adjusted, it is kept fixed in this study to isolate the effects of scheduling and workload contention on timing performance. In contrast, Priority-B exhibits a substantial increase in both average latency and execution-time dispersion, with the mean latency exceeding the deadline by more than a factor of three and the worst-case latency increasing by more than an order of magnitude.

Tables III and IV, together with Fig. 2, show a substantial degradation in timing performance as system contention increases. Under Priority-A, the inference task remains close to schedulable, with a mean latency of 8248.95 µs, a standard

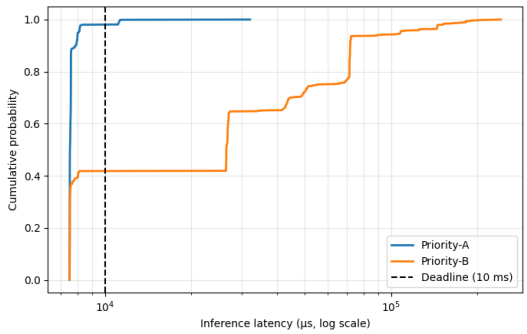


Fig. 2. Empirical cumulative distribution of inference latency under Priority-A and Priority-B scheduling. Priority-A remains tightly concentrated near the nominal execution range, whereas Priority-B shows substantially greater dispersion and heavier tail behavior under increased contention.

deviation of 1175.20 µs, and a deadline miss rate of only 1.50%. In contrast, under Priority-B, the mean latency increases to 36278.86 µs, the standard deviation rises sharply to 37157.75 µs, and the deadline miss rate reaches 59.00%, indicating a clear loss of real-time schedulability under contention. This behavior is also reflected in Fig. 2: the Priority-A curve rises steeply within a narrow latency range, whereas the Priority-B curve spreads across a much wider interval with a pronounced long-latency tail.

In addition to the increase in average latency, Priority-B demonstrates significantly higher execution-time variability, as reflected by the large standard deviation and extreme worst-case latency. While some executions are still completed within the nominal inference range, the presence of substantial long-tail delays indicates that the system is highly sensitive to interference from concurrent tasks.

These results reveal a clear schedulability boundary: while Priority-A operates close to operates in a near-schedulable region, Priority-B pushes the system into an unschedulable regime. From a real-time systems perspective, this degradation is consistent with the well-established effect of blocking and interference, where lower- or equal-priority activities can delay completion of time-critical work and increase response time unpredictability. In priority-driven systems, such interference can significantly reduce schedulability even when the computational workload itself is modest. In this study, the inference task is lightweight in isolation, yet its timing behavior deteriorates sharply when competing workloads are allowed to contend more aggressively for processor time.

The contrast between Priority-A and Priority-B therefore demonstrates that real-time correctness in embedded TinyML systems depends not only on model efficiency, but also on scheduling discipline and workload composition. Furthermore, the presence of large latency outliers shows that average-case timing is not sufficient for evaluating temporal correctness. For deployment in deadline-constrained systems, worst-case and tail-latency behavior must be treated as first-class design concerns.

TABLE V
OBSERVED STARVATION BEHAVIOR UNDER PRIORITY-C

Metric / Observation	**Priority-C Result**
Inference task priority	Lower than all background tasks
Background task behavior	Higher-priority sensing, communication, and I/O workloads
Inference period / deadline	10 ms / 10 ms
Target sample count	1000 inference activations
Experiment completion	Did not complete successfully
Valid timing statistics	Not available due to starvation
Observed failure symptom	Repeated watchdog triggers
Log evidence	Background tasks such as `IOLoad` frequently occupied processor time
System-level indication	Idle task unable to execute reliably
Interpretation	Unschedulable starvation configuration

D. Priority-C Starvation Behavior

The Priority-C configuration provides qualitative evidence of starvation under extreme scheduling interference. In this setup, the inference task was assigned a lower priority than all competing background activities, including sensing, communication, and I/O-related workloads. As a result, the inference task was repeatedly preempted by higher-priority tasks and was unable to make sufficient forward progress to complete the experiment.

From a systems perspective, this behavior is consistent with starvation in a priority-driven multitasking environment, where lower-priority work may be delayed indefinitely when higher-priority or continuously ready tasks dominate processor time. In this case, repeated watchdog triggers indicate that the system no longer provided a viable execution environment for periodic inference. Therefore, Priority-C is best interpreted not as a missing dataset, but as an experimentally observed failure mode that marks the practical limit of schedulability for the deployed inference workload.

The execution logs show that processor time was frequently occupied by background tasks such as `IOLoad`, while the inference task failed to make reliable forward progress. In particular, watchdog reports indicate that the idle task was unable to execute, which suggests sustained processor occupation by non-idle workloads. Under these conditions, the inference task could not be scheduled in time to maintain its intended periodic execution.

As summarized in Table V, no valid latency distribution or deadline-miss statistics could be collected for Priority-C. Unlike Priority-A and Priority-B, which produced measurable timing results, Priority-C represents a system breakdown scenario. This behavior demonstrates that, under sufficiently high interference, the system transitions from degraded timing performance to an unschedulable regime in which periodic inference execution can no longer be sustained.

E. Memory Usage

The memory footprint of the deployed TinyML inference system was evaluated to assess feasibility on resource-constrained embedded hardware. The results are summarized in Table VI.

The TensorFlow Lite Micro model requires a tensor arena of 40 KB, with an additional heap usage of approximately

TABLE VI
MEMORY USAGE OF TINYML INFERENCE SYSTEM

Component	Memory
Tensor arena	40 KB
Heap usage (observed)	14.4 KB
Total RAM footprint	≈ 54 KB
Minimum free heap	279 KB

14.4 KB, resulting in a total RAM footprint of about 54 KB. Despite this allocation, the system maintains a minimum free heap of 279 KB, indicating sufficient memory headroom.

These results confirm that the deployed model is lightweight and suitable for microcontroller-class devices. Memory usage remains stable throughout execution and does not contribute to system instability. Instead, as demonstrated in the timing results, scheduling interference and task contention are the dominant factors affecting real-time performance.

VII. DISCUSSION

A. Interpretation of Scheduling Effects

The experimental results demonstrate that real-time TinyML inference on the ESP32-S3 is feasible under controlled scheduling conditions, but highly sensitive to scheduler-level interference. Under Priority-A, the inference task remains in a near-schedulable region, achieving a mean latency below the 10 ms deadline with only a small fraction of deadline misses. However, occasional long-tail latency spikes still occur, indicating that favorable average-case timing does not by itself guarantee strict real-time correctness. This result shows that even when inference appears efficient in isolation, its real-time behavior must still be evaluated under the runtime conditions in which it will actually execute.

As system contention increases, scheduler-level interference becomes the dominant factor shaping execution behavior. Under Priority-B, both the average latency and timing variability increase sharply, and the deadline miss rate rises from 1.50% to 59.00%. This marks a clear transition from near-schedulable to unschedulable system behavior. From a real-time systems perspective, this result is consistent with the well-known effect of blocking and interference in priority-driven systems, where competing tasks can significantly inflate response time and reduce predictability. In this study, the inference task is computationally lightweight in isolation, yet still fails to meet timing requirements when contention is intensified.

The Priority-C configuration further highlights the practical limits of the system. Under extreme interference, the inference task was effectively starved and unable to make sufficient forward progress to complete the experiment, resulting in repeated watchdog triggers. This behavior is best interpreted as an experimentally observed failure mode rather than simply a missing dataset. It shows that beyond a certain interference threshold, the platform no longer provides a viable execution environment for periodic inference. In practical deployments, this type of behavior would be unacceptable because the system would not merely produce delayed predictions, but could stop producing inference results altogether.

Another important observation is that memory is not the limiting factor in this system. The deployed model occupies approximately 54 KB of RAM in total, while substantial free heap remains available throughout execution. This indicates that the observed degradation is primarily caused by scheduling contention rather than memory exhaustion. Consequently, the results reinforce the view that real-time TinyML deployment is fundamentally a co-design problem involving both model efficiency and system scheduling behavior.

B. Scheduling-Aware Mitigation Guidelines

The results suggest that improving TinyML timing behavior requires more than optimizing the neural network model itself. Since the same inference workload exhibits substantially different timing behavior under different priority assignments, mitigation should also be applied at the scheduling and system-design level. Based on the observed behavior across the evaluated priority configurations, several practical guidelines can be derived.

First, isolated inference latency should be measured before system integration and compared against the target deadline. This establishes the amount of available timing slack before sensing, communication, and storage workloads are introduced. If isolated inference latency is already close to the deadline, even moderate scheduler interference can cause deadline misses.

Second, inference priority should be assigned according to timing criticality. When inference is part of a periodic monitoring or control loop, it should not be placed below long-running or high-duty-cycle background tasks. The Priority-B and Priority-C results show that background workloads with higher priority can inflate response time or even prevent inference from making forward progress.

Third, background workloads should be bounded in both duration and frequency. Communication, sensing, and storage activities should be divided into smaller execution segments where possible, rather than executed as long uninterrupted bursts. This allows the scheduler to return control to the inference task more predictably and reduces the likelihood that background work delays inference across multiple periods.

Fourth, logging and file-system activity should be kept outside the inference critical path. In this study, SPIFFS-based logging supports persistent measurement collection, but storage operations can introduce additional timing variability if they directly compete with inference execution. Buffering measurements and deferring file writes to a lower-priority task can reduce interference with the periodic inference path.

Finally, starvation should be treated as a schedulability failure rather than ordinary performance degradation. The Priority-C configuration demonstrates that assigning inference below sustained high-priority background activity can make periodic inference execution unreliable. A practical mitigation strategy is therefore to protect the inference task from excessive scheduler-level interference, bound the execution demand of competing tasks, and validate the final priority configuration

under realistic multitasking conditions rather than only under isolated inference execution.

C. Energy Implications

Although this work does not directly measure energy consumption, the findings remain relevant for energy-constrained embedded systems. Increased interference, timing instability, and watchdog-triggered failures can prolong active processing time and reduce opportunities for predictable duty cycling. Therefore, improving timing predictability is not only a correctness issue, but also a practical systems concern for long-lived edge devices. In particular, a configuration that causes repeated deadline misses or starvation may keep the processor active longer than expected, reducing the effectiveness of low-power operation even if the neural network model itself is small.

D. Generalizability and Limitations

This study has several limitations. First, all experiments were executed on a single core to isolate scheduler interactions, so the results do not capture multicore partitioning effects. Second, the background workloads are simplified representations of sensing, communication, and I/O behavior; while realistic in structure, they do not span the full diversity of deployed embedded workloads. Third, the evaluation is limited to one platform, one model, and one deadline configuration, so the reported schedulability boundary should be interpreted as workload-specific rather than universal.

Nevertheless, the main trend is expected to generalize to other fixed-priority RTOS-based embedded systems. When periodic inference competes with higher-priority or high-duty-cycle background workloads, response time can become dominated by scheduler-level interference rather than isolated inference execution time. On multicore microcontrollers, this effect may be reduced by assigning inference and background workloads to separate cores, but shared memory, peripheral access, synchronization, and logging paths may still introduce interference. Future work should therefore evaluate additional TinyML models, alternative deadline settings, different microcontroller platforms, and multicore scheduling configurations.

VIII. Conclusion

This work shows that TinyML inference on a microcontroller-class platform can be memory-feasible and nominally fast, yet still fail to satisfy real-time requirements when exposed to realistic multitasking interference. The results demonstrate clear transitions from near-schedulable operation to unschedulable behavior and, under extreme conditions, to starvation. These findings support the need for tighter integration between machine learning design and real-time systems engineering in embedded AI deployments. Future work will focus on improving timing predictability through stronger temporal isolation between inference and background activities, reducing blocking from I/O and communication tasks, and exploring scheduling strategies beyond basic fixed-priority execution. Additional studies with longer experiment duration, alternative model architectures, and multicore deployment strategies may provide further insight into the robustness of embedded TinyML under practical operating conditions.

References

[1] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O'Reilly Media, 2020.

[2] Z. Zhang and J. Li, "A review of artificial intelligence in embedded systems," *Micromachines*, vol. 14, no. 5, p. 897, 2023.

[3] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, Sep. 1990.

[4] S. Davari and L. Sha, "Sources of unbounded priority inversions in real-time systems and a comparative study of possible solutions," in *Proceedings of the ACM SIGPLAN Workshop on Language, Compiler, and Tool Support for Real-Time Systems*, 1992, pp. 110–118.

[5] C.-K. Lin and B.-Y. Wang, "Analyzing freertos scheduling behaviors with the spin model checker," *arXiv preprint arXiv:2205.07480*, 2022.

[6] B. D. Miranda, R. S. de Oliveira, and A. Carminati, "Analysis of freertos overheads on periodic tasks," in *Proceedings of the Brazilian Symposium on Computing Systems Engineering (SBESC)*, 2020.

[7] H. Zhang, R. Zhang, and J. Sun, "Developing real-time iot-based public safety alert and emergency response systems," *Scientific Reports*, vol. 15, p. 29056, 2025.

[8] A. S. M. Mohsin and M. A. Muyeed, "Iot based smart emergency response system (sers) for monitoring vehicle, home and health status," *Discover Internet of Things*, vol. 4, p. 22, 2024.

[9] D. Borsatti, W. Cerroni, F. Tonini, and C. Raffaelli, "From iot to cloud: Applications and performance of the mqtt protocol," in *Proceedings of the 22nd International Conference on Transparent Optical Networks (ICTON)*, 2020, pp. 1–4.

[10] N. Saha, P. Paul, K. Ji, and R. Harika, "Performance evaluation framework of mqtt client libraries for iot applications in manufacturing," *Manufacturing Letters*, vol. 41, pp. 1237–1245, 2024.

[11] J. L. Reyes-Ortiz, D. Anguita, A. Ghio, L. Oneto, and X. Parra, "Human activity recognition using smartphones," 2013, [Dataset], UCI Machine Learning Repository. doi: 10.24432/C54S4K.

[12] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "Tensorflow lite micro: Embedded machine learning on tinyml systems," in *Proceedings of the 4th MLSys Conference*, 2021.